第20题 • 跨算子动态落盘调度

8专家与微型决策树
三成员数据集迭代报告

已完成测量的阶段记录 | 2026-10-30 | C++14 / CPU

方案将策略选择建模为成本敏感分类：读取合法可见状态和自身历史，小树输出专家编号，只执行被选中的专家。训练与部署均已实现；所有下表Score包含公开执行器实际测得的回调耗时。

当前本地最佳：tree_search_gl_1

785例平均Score为 67,222,897,889.92 μs，相对同批官方Naive下降 29.0629%。这是反复观察的开发/回归数据上的结果，尚无官方隐藏测试成绩，也不是全输入最优证明。

成员 / 版本	例数	平均Score / μs	比Naive下降	原逐例分均值 / 10
Vanius0525 / 0.1.0	209	157,477,620.08	35.87%	3.4526
cyanneko / 0.6.0	48	7,064,788,139.59	47.66%	3.5791
liunianwww / 0.1.0	528	99,238,545,814.94	28.89%	2.5011

每例串行测量3次，先取逐例算术均值，再对样例取算术均值。三人数据分别为209、48、528例；Vanius0525的17个合法边界例全部计入。空member2/member3未计入。

同批方法	平均完整决策时间 / μs每例	miss总数
原NN	1,233.33	397,224
父树	1,372.02	463,040
新树	1,532.47	614,457

新树的完整决策时间和miss数有所增加；I/O降低后，包含这些代价的总Score仍更低。小模型推理与专家执行的取舍由完整计时决定。

两种指标分别保留

仓库主归一化汇总：先对每成员原逐例贡献求和、不做50分总封顶，再对3个成员等权平均，本轮为 737.9838。原封顶50分汇总在三份集合上均已饱和为50，难以区分方案。主归一化分与raw平均Score是不同指标，后者越低越好。

数据固定提交：e119c2326e198922243306eac062785420ed8156。
套件指纹：a176758680d7c6a99805cfb1eeeff9786314d8309db7c0e80e269843880e8ce1

02 / 选择哪些决策方法

8种专家提供互补的容量、复用、代价与多算子管理能力。文献思想经过本题接口适配，不能直接继承原论文的收益或竞争比。编号0至7才是在线分类标签；评测mode只是完整调度器的离线编号。

ID / 方法	本题实现与用途	借鉴与严格性边界
0 最大分区	跨算子按驻留字节降序释放空间。	Velox官方落盘文档[1]；快速启发式。
1 精确最小写出覆盖	候选≤10时枚举子集，满足缺口且写出字节最少；更大时回退。	当前规划子问题严格最优；不保证后续总代价。
2 观测最近访问	按接口实际观测的访问时刻选择。	LRU思想；普通hit无回调，非完整LRU。
3 观测热度密度	按(1+衰减热度)/字节评价保留价值。	LHD单位空间价值[2]；未实现原概率模型。
4 恢复代价密度	比较[s/W+q·(s/R+32)]/s，兼顾读写。	代价感知缓存[3]；q、32是启发尺度。
5 观测GDS	维护通胀值及H=L+c/s，更新可见访问。	GreedyDual-Size[3]；原竞争界不直接适用于本题。
6 阶段与算子软配额	跨算子分配软预算，BUILD和PROBE分别规划。	DuckDB多算子内存管理[4]；本实现为启发式。
7 自适应反馈控制器	ghost回访反馈切换规则，结合配额和保守next_op。	本项目前版；参考LeCaR反馈思想[5]。

跨算子、阶段与动态感知

候选分区覆盖全部算子，而非只在当前算子内找牺牲者。BUILD关注当前新增空间，PROBE关注恢复与回访；观测到的miss率、ghost回访、热度及相位持续变化，并输入下一次选择。最终树未必使用全部8类，但候选库保留全部方法。

缺口D=max(0,d-(M-U))。M是容量，U是当前使用量；溢出时d为新增元组大小，miss时为完整目标分区大小，联合规划时为两分区大小之和。

03 / 本项目贡献及局部证明

贡献一：必落盘集合与学习选择器共用

令locked为当前请求必须保留的总字节：BUILD含目标已有数据和新增元组，PROBE含目标分区；若另有必须保护的驻留分区，再计入其大小且不重复。任意候选x若s(x)>M-locked，则它无法与locked共存，必定需要落盘。

令F为所有上述必落盘分区。若Σ(x∈F)s(x)≥D，只落盘F就满足容量；任何合法方案又必须包含F，故其立即写出量至少为Σs(x)。返回F达到该下界。该保证只针对已固定需求及保护条件的立即写出量，不能推出完整未来Score最优。

例：M=100、必须保留70，驻留候选40与20、缺口30。40无法与70共存，必落盘；它单独覆盖缺口，于是无需运行复杂子集搜索。若所有候选都不构成必落盘，则继续使用被选专家。

贡献二：按决策事件频率纠正训练权重

有限采样会低估高频miss/overflow输入。对文件f、回调类k，记录完整轨迹事件数T及采样数S，给每条状态权重T/S。叶子选择argmin_e Σw_i·Loss(C_i,e)，分裂以左右叶子的最小成本下降为目标；不用普通分类准确率代替成本。

当前父模型使用640份train采样的15,801个状态。新三成员840份train又采集20,764个状态；无决策事件的输入仍在全程评测中保留。有限窗口与重叠窗口只提供近似标签，不是完整Score的无偏估计。

贡献三：沿树路径付出特征成本

当前模型深度4、29节点、15叶子。阈值展开为内嵌if/else，最多4次比较；cheap特征按路径懒计算，避免先汇总全部分区统计。无需在线模型文件或推理库。

这不表示整个回调O(1)：公开getSystemState复制、专家候选收集和规划仍可能O(Q)。真正收益必须由实际回调时间和完整Score检验。

贡献四：完整train Score闭环微调

以小树为起点，只在训练特征的邻近阈值或叶子专家上做小范围变更，保持容量公共层不变；在840份完整train轨迹上串行计算实际Score筛选。随后冻结候选到785例eval复测。此为本项目实现贡献，尚未主张这些组合首次见于文献。

04 / 在线运行与训练边界

触发时机	实际行为
首次及后续PROBE miss	每次重新分类，再更新当前可观测反馈，执行所选专家并把当前目标LOAD放在最后。
INSERT内存溢出	每次重新分类，排除当前插入目标，跨算子落盘。
周期回调看到下一INSERT将溢出	提前分类与释放空间，减少之后的压力。
周期回调看到下一PROBE	不进行树分类；仅在空闲空间或强制集合条件下保守预载，并更新部分可见历史。
普通hit	无对应回调，不编造完整访问时间线。

next_op只是一条已知操作

只使用接口给出的next_op，做保守的目标保护、无需额外落盘的联合加载等。没有下一步之后的未来访问流，也看不到真实miss罚分p、总操作数N或周期B。若接口中的pair_fits历史特征有同分区重复计数，它仅为训练特征，容量判断始终由精确动作层完成。

离线标签与信息隔离

教师复制同一状态，让8个专家分别执行首动作，再由共同续行策略推进。成本含 Δ IO+p • Δ miss及测得的决策时间。默认窗口384步，BUILD可延伸到当前算子SWITCH之后128步、最多2048步。未来流和真实p仅用于离线评分，不进入在线特征。

训练检查了输入字节、原始操作流和按key%P归一的操作流，避免同一流跨train/eval；标签实际只来自train。用于比较或反复查看的eval已是开发/回归数据，不能称为未见盲测。

时间与容量边界

完整Score=IO时间+p • miss次数+实际决策时间。保留官方计时范围，包含状态获取；不把路由比较数当成实测速度。所有动作独立核对容量、插入目标排除、LOAD阶段、miss末动作及lazy特征预测一致性。

05 / 迭代证据与复现

旧固定568例：NN平均约96,578,213,902.12 μs，第一轮树94,273,432,606.00，加入强制集合94,247,452,742.66，事件加权cheap树92,922,068,139.66。后者比旧快照Naive下降28.7795%。旧数据与新785例分母不同，不直接比较两者平均值大小。

新785例：增加Vanius0525的209例，并纳入cyanneko 0.6.0的48例。新train重拟合模型曾在cyanneko上达到50.06%降幅，但全库差于父树；因此未仅因一份集合改善就替换。全部负结果与逐例坏例保留在结果文件。

完整train微调共测试38个新变异/组合（16+16+6）。选中tree_search_g1_1：训练均值比父树下降1.619%，785例eval同批下降0.448%；Vanius单集原始均值却升高约10.78%，明确保留。训练最好组合下降2.358%，eval反而变差，未采用。

随后单独验证数组复用：785例动作完全一致，3次完整回调D均更慢，平均由1,525.31升至1,592.81 μs（增加4.43%），未采用。该轮保留版本Score为67,222,897,882.75 μs。

尾部须结合绝对成本与IO看：父树某次最差241倍来自无IO、无miss的小例，约182 μs对照0.75 μs；而未采用模型曾在hot-small例引发约440万μs真实IO。两个现象都保留，含义不同。

单文件接入与复测

交付文件：design/MySpillScheduler_tree.cpp。定义MySpillScheduler、无main，接入公开C++14框架。编译运行：python design/tools/run.py --policy tree --input <正式输入文件>。原rule/NN版本保留便于比较。

训练、生成、评测：portfolio_iter_20261029/README.txt。数据在GitHub datasets分支的独立worktree，固定e119c23；原数据、截图、二进制及临时快照不混入源码分支。结果按suite_id、源码SHA、重复数绑定，重新测量CPU值会波动。

当前交付源码SHA-256：1e719564a213c59bab59fd162a3bf233ae0564d14c29a96766fdd340436817fd

文献与业界来源（思想借鉴）

[1] Velox: Spilling

[2] Beckmann等, LHD, NSDI 2018

[3] Cao与Irani, GreedyDual-Size, USITS 1997

[4] Kuiper等, DuckDB多算子内存管理, PVLDB 18

[5] Vietri等, LeCaR, HotStorage 2018

[6] Optimal Sparse Decision Trees, NeurIPS 2019

小树采用成本敏感贪心CART，不是文献[6]的全局最优树求解器。更多来源、实现差异及保证边界见catalog.json、research_sources.json。